ChatGPT

Modular Agentic System Scaffolding

This document describes a modular, database-driven architecture for running AI agents with flexible thinking strategies and a persistent knowledge base (KB). It builds on the previous discussion by detailing how to organise agents, tools, prompts and strategies in the database, and it shows how to incorporate *notes* into the KB with a simple boolean flag. The goal is to make the system easy to adapt – you can select models, apply stored prompts, choose thinking methods and assemble toolsets per phase without changing core code.

1. Architecture overview

The system uses three major components:

1. **ParadeDB / Postgres** – stores all structured data: models, tools, prompts, agent kits, strategies and run history. It also holds the KB for documents and notes. ParadeDB adds BM25 + vector search over text fields.
2. **Neo4j** – maintains a provenance graph showing which plans, tasks and tools were used to produce each artefact. It mirrors entries from Postgres but keeps them in graph form for inspection and querying.
3. **Prefect** – executes DAG plans emitted by the planner. Each plan is a flow; each node is a task that calls a tool or runs code. Prefect enforces timeouts, RBAC and side-effect isolation.

Agents are not hard-coded classes. Instead, there is a single **agent runner** that interprets an `AgentRequest` JSON, looks up the relevant kit in the database and then executes each phase with the chosen strategy. Strategies implement common reasoning patterns such as ReAct, RAP (plan first then act), CodeAct (code generation with tests) and Reflection【417213429674161†L150-L163】.

2. Database schema (core tables)

The following tables support modularity. Namespaces (`dev`, `cfg`, `run`, `memory`) keep logical groups separate. Only the key columns are shown for brevity:

table (schema)	columns / purpose
`dev.tools`	`id`, `slug`, `schema`, `instructions`, `exec_ref`, `caps` – definitions for tools, including function signatures and capabilities.
`dev.toolsets`	`id`, `slug`, `tool_ids int[]`, `policy` – named bundles of tools, e.g. `planning_minimal`, `research_std`.
`dev.prompts`	`id`, `slug`, `role` (`system`/`user`), `content`, `meta` – store reusable prompt templates.

table (schema)	columns / purpose
`cfg.models`	`id`, `slug`, `uri`, `family`, `dims`, `caps` – model registry with metadata (e.g. `qwen3`, `qwen3-code`, `gemini-2.0-pro`).
`dev.strategies`	`id`, `slug`, `type`, `params` – definitions for thinking patterns such as `react`, `rap`, `codeact`, `reflection`, `mctis`.
`dev.agent_kits`	`id`, `slug`, `default_prompt_slug`, `default_toolset_slug`, `default_strategy_slug`, `allowed_models text[]`, `params` – encapsulate a set of defaults and phase-specific overrides.
`run.sessions`	`id`, `request JSONB`, `state`, `created_at` – top-level record for a single agent run.
`run.plans`	`id`, `session_id`, `plan JSONB`, `status` – holds the DAG spec produced by the planner before execution.
`run.nodes`	`id`, `plan_id`, `kind`, `spec JSONB`, `status`, `artifact_uuid` – individual nodes of a plan (tool calls, code tasks, etc.).
`run.artifacts`	`uuid`, `kind`, `uri`, `meta` – outputs produced by the run; stored on disk or object storage.
`memory.items`	`uuid`, `kind`, `body`, `summary`, `is_note boolean`, `meta_id`, `related_refs uuid[]` – persistent entries such as documents, code, notes. Set `is_note` to `true` for note entries.
`memory.meta`	`id`, `key`, `value`, `item_uuid` – flexible metadata associated with a KB item (tags, timestamps, authorship). Each meta row links back to the parent item.

Searching the KB

ParadeDB automatically indexes `memory.items.body` and `memory.items.summary` for lexical and vector search. To find notes related to a goal or specific UUIDs:

- Use lexical filters on `is_note` to isolate notes.
- Query `related_refs` to retrieve notes linked to a given document or artefact.
- Apply vector similarity search on the `body` or `summary` fields to surface semantically relevant entries.

3. Agent request contract

Agents are invoked by sending an `AgentRequest` JSON to the runner service. The runner resolves defaults from the agent kit and applies any user-provided overrides. A typical request looks like this:

```
{
  "goal": "Draft a motion outline for a shipping dispute",
  "context_refs": ["kb:8f2…", "kb:b1c…"],
```

```
  "agent_kit": "planner_v1",           // kit slug
  "overrides": {
    "model": "qwen3",                // pick a model family
    "toolset": "planning_minimal",    // restrict tool access
    "strategy": "rap",                // default reasoning pattern
    "budgets": { "tokens": 40000, "tool_calls": 12, "wall_time_s": 180 },
    "flags": { "explore": true, "reflect": true, "code_fallback": true }
  },
  "constraints": { "citations": true, "deadline_s": 600, "privacy":
"local_only" },
  "outputs": { "format": "markdown", "deliverables": ["outline", "citations"] }
}
```

The runner proceeds through phases (router, planner, discovery, research, execution, responder, auditor, write-back). For each phase it selects the appropriate strategy from the `strategy_map` in the kit's `params`. Budgets and models can be updated at run-time. The `context_refs` array may include UUIDs of notes or documents from the KB.

4. Notes in the knowledge base

4.1 Ingesting a note

When a user or agent creates a note, follow these steps:

1. **Generate a UUID** (v4) for the note.
2. **Insert into** `memory.items` with `kind = 'text'`, the raw note content in `body`, an optional short summary in `summary`, and `is_note = true`. If the note relates to other items, populate `related_refs` with their UUIDs.
3. **Insert metadata** into `memory.meta` as key–value pairs. Common keys include `tags` (array), `author`, `created_at` and `source`. Each meta row stores `item_uuid` to reference its parent.
4. **Compute embeddings** on the `body` and store them in ParadeDB's vector index. If you already use a background worker for embeddings, enqueue the note UUID for processing.
5. **Return the UUID and summary** to the caller. The summary may be auto-generated by the agent runner when the note is created.

Example SQL

```
INSERT INTO memory.items (uuid, kind, body, summary, is_note, related_refs)
VALUES (
  :uuid,
  'text',
  :body,
  :summary,
  true,
  :related_refs
);
```

```
INSERT INTO memory.meta (item_uuid, key, value)
VALUES
  (:uuid, 'tags', to_jsonb(ARRAY['project-alpha','meeting'])),
  (:uuid, 'author', 'george'),
  (:uuid, 'created_at', now()::text);
```

4.2 Retrieving notes

To fetch notes relevant to the current task:

1. **Filter on** `is_note = true` to avoid mixing notes with other KB items.
2. **Apply semantic search** using ParadeDB's `match` function on `body` or `summary` with a query. Combine with a lexical filter on `related_refs` if the note should be linked to particular UUIDs.
3. **Sort by relevance** and limit the number returned. You can also filter by tags or date through the `memory.meta` table.

Pseudo-SQL query

```
SELECT i.uuid, i.summary, i.body
FROM memory.items i
LEFT JOIN memory.meta m ON i.uuid = m.item_uuid
WHERE i.is_note = true
  AND (m.key = 'tags' AND m.value \? :desired_tag)
ORDER BY match(i.summary, :query) DESC
LIMIT 5;
```

4.3 Using notes during reasoning

When the agent runner receives an `AgentRequest`, it can automatically pull relevant notes into the `context_refs` by querying the KB using the goal description as the search query. These references are added to `context_refs` and passed to the planner and responder. In the final output, citations can be generated by referencing the UUIDs of notes used.

5. Prefect integration

Prefect flows are created from the planner's DAG spec. Each node of the spec includes a `kind` (`tool_call`, `code_task`, `search_task`, etc.), the tool slug or code reference, inputs, budgets and an `artifact_contract` describing expected outputs. Prefect tasks run side-effecting actions and record artefacts in `run.artifacts`. For notes, you may add a small `note_writer` tool that inserts into the KB and returns the new UUID.

Example tool definition for notes

```
{
  "slug": "write_note",
  "schema": {
    "type": "object",
    "properties": {
      "body": {"type": "string"},
      "summary": {"type": "string"},
      "related_refs": {"type": "array", "items": {"type": "string"}}
    },
    "required": ["body"]
  },
  "instructions": "Persist a note in memory.items and memory.meta; return the
new UUID.",
  "exec_ref": "prefect.functions.write_note",
  "caps": {"side_effect": true}
}
```

The corresponding Prefect task `write_note` would implement the ingestion steps outlined in §4.1.

6. Example ingestion and usage flow

Below is a high-level pseudo-code for creating and retrieving notes within the agent runner:

```
def create_note(body: str, summary: str | None, related_refs: list[str]) -> str:
    uuid = generate_uuid()
    db.insert_into_memory_items(uuid=uuid, kind='text', body=body,
summary=summary or '', is_note=True, related_refs=related_refs)
    db.insert_meta(uuid, 'created_at', str(datetime.utcnow()))
    if related_refs:
        db.insert_meta(uuid, 'related_refs_count', len(related_refs))
    embedding_worker.enqueue(uuid)  # compute vector asynchronously
    return uuid

def search_notes(query: str, tags: list[str] | None = None, limit: int = 3) ->
list[Note]:
    sql = "SELECT i.uuid, i.summary, i.body FROM memory.items i"
    sql += " LEFT JOIN memory.meta m ON i.uuid = m.item_uuid"
    sql += " WHERE i.is_note = true"
    if tags:
        sql += " AND m.key = 'tags' AND m.value \? :tags"
    sql += " ORDER BY match(i.summary, :query) DESC LIMIT :limit"
    return db.query(sql, tags=tags, query=query, limit=limit)
```

```
# Example usage in a responder phase
notes = search_notes(agent_request.goal, tags=['project-alpha'])
context_refs.extend([f"kb:{note.uuid}" for note in notes])
```

7. Additional recommendations

- **Normalise all text** before embedding to ensure consistent vector representations (lowercase, remove HTML). The vector store should be updated whenever the `body` or `summary` changes.
- **Provide a user interface** for viewing and editing notes. A simple web page listing `summary`, `created_at`, `tags` and linking to full contents by UUID helps maintain visibility into the KB.
- **Set retention policies** for notes (e.g. archive or delete after a certain period) by adding an `expires_at` meta key. Scheduled jobs can act on these flags.
- **Enforce access control** if multiple users are involved. You may add an `owner` column to `memory.items` and check it before retrieval or modification.
- Use **Reflection or R★ patterns selectively**. Reflection (self-evaluation with one or two passes) improves reliability【417213429674161†L150-L163】. More involved tree-search patterns (R★/MCTS) should be reserved for high-stakes audits; they consume more resources.

By organising your data in this way and keeping the agent runner logic simple, you gain the flexibility to experiment with different strategies, models and toolsets. Notes become first-class citizens in the KB, easily referenced, searched and updated.
